

SIMATS ENGINEERING

ASSESSMENT TOOL 3 - LOW (ASSIGNMENT - 2)

COURSE CODE: CSA09

**COURSE NAME: Programming in
Java**

COURSE OUTCOME COVERED

CO3: *Build multithreaded Java programs by applying thread priorities, synchronisation, and inter-thread communication mechanisms to achieve concurrent program execution and use exception handling techniques (BL3,BL4)*

Assessment Tool Weightage: 10%

QUESTIONS: 1

Total Marks: 50

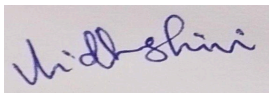
ANNEXURE A

ASSIGNMENT - 2

Code of Conduct:

*I Vidhushini.T.S (192511061) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:



ANNEXURE

ASSIGNMENT – 2

10: Bank Loan Application Processing System

A financial institution receives loan applications through several branches and online portals. Each application must be validated for applicant information, income, requested amount, credit score, and repayment period. Invalid values, missing information, and insufficient eligibility may generate different exceptions. Since thousands of applications are received simultaneously, the bank wants to process applications using multiple threads.

Develop a Java application that performs the following:

- a) Create classes for Applicant, LoanApplication, and LoanProcessor.
- b) Validate applicant name, income, requested loan amount, and credit score.
- c) Handle built-in exceptions such as `NumberFormatException`, `NullPointerException`, and `IllegalArgumentException`.
- d) Create user-defined exceptions such as `InvalidIncomeException`, `LoanLimitExceededException`, and `LoanNotEligibleException`.
- e) Use `try-catch-finally` during application processing.
- f) Create multiple threads for processing applications from different branches.
- g) Use thread priorities for urgent loan applications.
- h) Use synchronization while updating the common approved-loan database.
- i) Implement handling for uncaught exceptions and generate a final approval/rejection report.

1. RUBRICS FOR CALCULATION

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	15%	Clearly understands the problem and requirements	Understands most requirements	Understands basic requirements	Has difficulty understanding requirements
Application of Java Concepts	20%	Applies relevant Java concepts correctly and effectively	Applies concepts with minor errors	Applies concepts with some limitations	Applies concepts incorrectly
Program Design & Logic	15%	Logical, modular, and efficient design	Well-structured design with minor issues	Basic design with limited logic	Poor or unclear design
Program Functionality	20%	Fully functional and produces correct results	Mostly functional with minor errors	Partially functional with some errors	Incomplete or non-functional
Error Handling	10%	Handles errors and invalid inputs effectively	Handles major errors appropriately	Handles limited error conditions	Provides little or no error handling
Testing & Validation	10%	Tests multiple cases and validates results	Tests major cases adequately	Performs limited testing	Performs minimal or no testing
Code Quality & Documentation	5%	Clean, readable, and well documented	Good code with adequate documentation	Basic code and documentation	Poorly organized and documented
Output & Presentation	5%	Clear output with effective analysis and presentation	Good output and presentation	Basic output and presentation	Unclear or incomplete output

Program:

```
import java.util.ArrayList;
import java.util.List;
/* SUBDIVISION D: User-defined exceptions */
class InvalidIncomeException extends Exception {
    public InvalidIncomeException(String message) {
        super(message);
    }
}
class LoanLimitExceededException extends Exception {
    public LoanLimitExceededException(String message) {
        super(message);
    }
}
class LoanNotEligibleException extends Exception {
    public LoanNotEligibleException(String message) {
        super(message);
    }
}
/* SUBDIVISION A: Class Applicant */
class Applicant {
    private String name;
    private double income;
    private int creditScore;
    public Applicant(String name, double income, int creditScore) {
        this.name = name;
        this.income = income;
        this.creditScore = creditScore;
    }
    public String getName() {
        return name;
    }
    public double getIncome() {
        return income;
    }
    public int getCreditScore() {
        return creditScore;
    }
}
/* SUBDIVISION A: Class LoanApplication */
class LoanApplication {
    private String applicationId;
    private Applicant applicant;
    private double requestedAmount;
    private int repaymentPeriodMonths;
    private boolean isUrgent;
```

```

    public LoanApplication(String applicationId, Applicant applicant, double requestedAmount,
int repaymentPeriodMonths, boolean isUrgent) {
        this.applicationId = applicationId;
        this.applicant = applicant;
        this.requestedAmount = requestedAmount;
        this.repaymentPeriodMonths = repaymentPeriodMonths;
        this.isUrgent = isUrgent;
    }
    public String getApplicationId() {
        return applicationId;
    }
    public Applicant getApplicant() {
        return applicant;
    }
    public double getRequestedAmount() {
        return requestedAmount;
    }
    public boolean isUrgent() {
        return isUrgent;
    }
}
/* SUBDIVISION H: Common approved-loan database with synchronization */
class ApprovedLoanDatabase {
    private static List<String> approvedLoans = new ArrayList<>();
    private static List<String> rejectedLoans = new ArrayList<>();
    public synchronized static void addApproved(String details) {
        approvedLoans.add(details);
    }
    public synchronized static void addRejected(String details) {
        rejectedLoans.add(details);
    }
    public static void printReport() {
        System.out.println("\n=== FINAL APPROVAL/REJECTION REPORT ===");
        System.out.println("APPROVED APPLICATIONS:");
        if (approvedLoans.isEmpty()) {
            System.out.println(" None");
        } else {
            for (String app : approvedLoans) {
                System.out.println(" - " + app);
            }
        }
        System.out.println("REJECTED APPLICATIONS:");
        if (rejectedLoans.isEmpty()) {
            System.out.println(" None");
        } else {
            for (String app : rejectedLoans) {
                System.out.println(" - " + app);
            }
        }
    }
}

```

```

    }
}
}
/* SUBDIVISION A & F: LoanProcessor class implementing Runnable for multi-threading */
class LoanProcessor implements Runnable {
    private String branchName;
    private List<LoanApplication> applications;
    public LoanProcessor(String branchName, List<LoanApplication> applications) {
        this.branchName = branchName;
        this.applications = applications;
    }
    /* SUBDIVISION B, C, D, E: Validation, Exception Handling, and try-catch-finally */
    public void validateAndProcess(LoanApplication app) throws Exception {
        try {
            if (app.getApplicant() == null) {
                throw new NullPointerException("Applicant data is missing.");
            }
            Applicant applicant = app.getApplicant();
            if (applicant.getName() == null || applicant.getName().trim().isEmpty()) {
                throw new IllegalArgumentException("Applicant name cannot be empty.");
            }
            if (applicant.getIncome() <= 0) {
                throw new InvalidIncomeException("Income must be positive.");
            }
            if (app.getRequestedAmount() <= 0) {
                throw new IllegalArgumentException("Requested loan amount must be positive.");
            }
            if (app.getRequestedAmount() > applicant.getIncome() * 10) {
                throw new LoanLimitExceededException("Requested loan exceeds 10x annual
income limit.");
            }
            if (applicant.getCreditScore() < 650) {
                throw new LoanNotEligibleException("Credit score below minimum requirement of
650.");
            }
            ApprovedLoanDatabase.addApproved("[ " + branchName + " ] ID: " +
app.getApplicationId() + " | Name: " + applicant.getName() + " | Amount: $" +
app.getRequestedAmount());
            System.out.println("SUCCESS: Application " + app.getApplicationId() + " approved
by " + branchName);
        } catch (NullPointerException | IllegalArgumentException e) {
            ApprovedLoanDatabase.addRejected("[ " + branchName + " ] ID: " +
app.getApplicationId() + " | Reason (Built-in Exception): " + e.getMessage());
            System.out.println("FAILED: Application " + app.getApplicationId() + " rejected -> " +
e.getMessage());
        } catch (InvalidIncomeException | LoanLimitExceededException |
LoanNotEligibleException e) {

```

```

        ApprovedLoanDatabase.addRejected("[ " + branchName + "] ID: " +
app.getApplicationId() + " | Reason (User Exception): " + e.getMessage());
        System.out.println("FAILED: Application " + app.getApplicationId() + " rejected -> " +
e.getMessage());
    } finally {
        System.out.println("Log: Processing attempt completed for Application ID " +
app.getApplicationId());
    }
}
@Override
public void run() {
    for (LoanApplication app : applications) {
        try {
            validateAndProcess(app);
        } catch (Exception e) {
            System.out.println("Unexpected exception caught: " + e.getMessage());
        }
    }
}
}

public class Main {
    public static void main(String[] args) {
        /* SUBDIVISION I: Uncaught Exception Handler */
        Thread.setDefaultUncaughtExceptionHandler((thread, throwable) -> {
            System.out.println("Uncaught Exception in thread " + thread.getName() + ": " +
throwable.getMessage());
        });
        /* Sample Branch 1 Applications */
        List<LoanApplication> branch1Apps = new ArrayList<>();
        branch1Apps.add(new LoanApplication("APP101", new Applicant("Alice", 60000, 720),
150000, 24, false));
        branch1Apps.add(new LoanApplication("APP102", new Applicant("Bob", -5000, 700),
50000, 12, false));
        branch1Apps.add(new LoanApplication("APP103", new Applicant("Charlie", 40000,
580), 30000, 12, false));
        /* Sample Branch 2 Applications (Contains Urgent Loan) */
        List<LoanApplication> branch2Apps = new ArrayList<>();
        branch2Apps.add(new LoanApplication("APP201", new Applicant("Diana", 80000, 750),
1000000, 36, true));
        branch2Apps.add(new LoanApplication("APP202", new Applicant("Eve", 90000, 800),
200000, 48, true));
        /* SUBDIVISION F: Creating multiple threads for different branches */
        Thread t1 = new Thread(new LoanProcessor("Branch-North", branch1Apps),
"NorthBranchThread");
        Thread t2 = new Thread(new LoanProcessor("Branch-South", branch2Apps),
"SouthBranchThread");
        /* SUBDIVISION G: Thread priorities for urgent loan applications */
        t1.setPriority(Thread.MIN_PRIORITY);
    }
}

```

```

        t2.setPriority(Thread.MAX_PRIORITY);
        t1.start();
        t2.start();
        try {
            t1.join();
            t2.join();
        } catch (InterruptedException e) {
            System.out.println("Main thread interrupted.");
        }
        /* SUBDIVISION I: Generate final approval/rejection report */
        ApprovedLoanDatabase.printReport();
    }
}

```

Output:

```

SUCCESS: Application APP101 approved by Branch-North
FAILED: Application APP201 rejected -> Requested loan exceeds 10x
annual income limit.
Log: Processing attempt completed for Application ID APP201
Log: Processing attempt completed for Application ID APP101
SUCCESS: Application APP202 approved by Branch-South
Log: Processing attempt completed for Application ID APP202
FAILED: Application APP102 rejected -> Income must be positive.
Log: Processing attempt completed for Application ID APP102
FAILED: Application APP103 rejected -> Credit score below minimum
requirement of 650.
Log: Processing attempt completed for Application ID APP103

=== FINAL APPROVAL/REJECTION REPORT ===
APPROVED APPLICATIONS:
    - [Branch-North] ID: APP101 | Name: Alice | Amount: $150000.0
    - [Branch-South] ID: APP202 | Name: Eve | Amount: $200000.0
REJECTED APPLICATIONS:
    - [Branch-South] ID: APP201 | Reason (User Exception): Requested loan
exceeds 10x annual income limit.
    - [Branch-North] ID: APP102 | Reason (User Exception): Income must be
positive.
    - [Branch-North] ID: APP103 | Reason (User Exception): Credit score
below minimum requirement of 650.

=== Code Execution Successful ===

```